# W4153 – Cloud Computing

## Lecture 6:

## REST (4), Composition, OAuth2, Functions

# *Contents*

# Contents

- Team presentations
- Project status/next sprint
- REST continued
  - Response codes/status codes
  - 201 Created (https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/201)
  - 202 Accepted -- Asynchronous invocation (https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/202)
- Service composition
  - Concepts
  - Orchestration, Choreography
  - Synchronous, Asynchronous, event driven
- Function-as-a-Service
  - Concepts
  - My application scenario
  - Show code and walkthrough
- ~~OAuth2~~
  - ~~Concepts and scenarios~~
  - ~~JWT Tokens~~
  - ~~Access token and refresh token; auto-refresh~~
- 12 Factor Applications -- Overview
- Next sprint

# *Team Presentations*

# Team Presentations and Updates

| | |
|---|---|
| Team: | ”Unknown” |
| Focal point: | He Qian SHEN |

# *Project Status/Sprints*

# Project Status and Sprints

- But first, my philosophy … …

# *REST Continued*

# HTTP Status Codes

- There are many, many HTTP response status codes.
  - I cannot remember most of them.
  - Also, I almost never see any of them.
  - But, that is not surprising. Normally the UI or client application handles an error status gracefully and "translates" into an application specific exception or easily understood error page.
- There status codes fall into five categories
  - Informational responses (100 – 199)
  - Successful responses (200 – 299)
  - Redirection messages (300 – 399)
  - Client error responses (400 – 499)
  - Server error responses (500 – 599)
- There are several good sources of information
  - https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status
  - https://www.restapitutorial.com/httpstatuscodes
  - https://www.w3schools.com/tags/ref_httpmessages.asp

# Which Ones do I Care about for your Project

- Once you have learned how to catch exceptions and return status codes for a few conditions, you understand the pattern for all the conditions.
- Many are subject to interpretation. My primary concern is that you
  - Are *consistent* over all of your microservices.
  - You document the status codes in Open API documentation.
- The ones I recommend/care about are:

| | |
|---|---|
| 200 OK | The request is OK (this is the standard response for successful HTTP requests) |
| 201 Created | The request has been fulfilled, and a new resource is created |
| 202 Accepted | The request has been accepted for processing, but the processing has not been completed |

- We will cover these today.

# Which Ones do I Care about for your Project

| | |
|---|---|
| 400 Bad Request | The request cannot be fulfilled due to bad syntax |
| 401 Unauthorized | The request was a legal request, but the server is refusing to respond to it. For use when authentication is possible but has failed or not yet been provided |
| 403 Forbidden | The request was a legal request, but the server is refusing to respond to it |
| 404 Not Found | The requested page could not be found but may be available again in the future |
| 405 Method Not Allowed | A request was made of a page using a request method not supported by that page |
| 409 Conflict | The request could not be completed because of a conflict in the request |
| 412 Precondition Failed | The precondition given in the request evaluated to false by the server |
| 418 I am a teapot | Just kidding. |
| 429 Enhance Your Calm | Just kidding |
| 422 Unprocessable Entity | The request was well-formed (i.e., syntactically correct) but could not be processed |
| 428 Precondition Required | |
| 500 Internal Sever Error | Error in the server that was not caught and handled properly/gracefully. My go to default. |
| 501 Not Implemented | The server either does not recognize the request method, or it lacks the ability to fulfil the request. Usually this implies future availability (e.g., a new feature of a web-service API). |

For eTag processing, which we covered last week.

# 201 – Created

- A POST on a collection /persons is how you create a new resource (person). The URL of the new resource is something lie /persons/dff9.
- The *client* has not way of knowing/being certain what the URL is.
- This is probably some type of primary key. There are two types of primary key
  - Natural key – the key derives from the data.
  - Surrogate key – the key is an arbitrary, generated value not easily derived from the data.
- Even
  - If the client/user “thinks” that they know the algorithm, they cannot be sure.
  - Moreover, this is “side information” and the interface is not not HATEOAS/self-describing.
- So,
  - A successful POST returns 201
  - With a *link header* containing the URL of the resource.

# 201 – Created

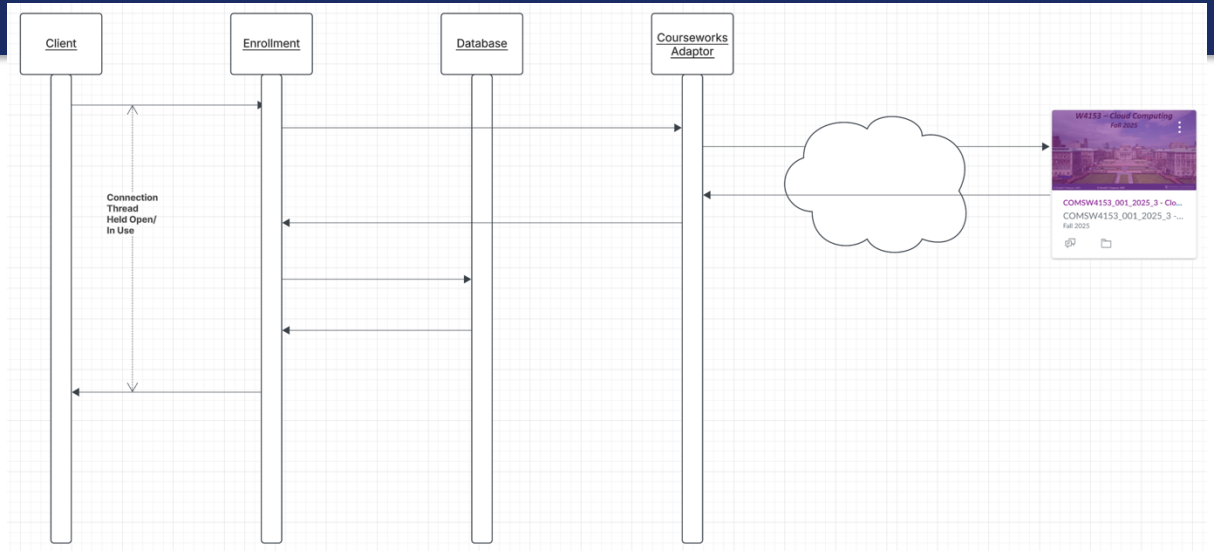
```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "firstName": "Brian",
  "lastName": "Smith",
  "email": "brian.smith@example.com"
}
```

Microservice

- Validates the request.
- Computes a primary key, perhaps relying on a key created by the data service.
- Persists the data.
- Returns 201 created with a location header.

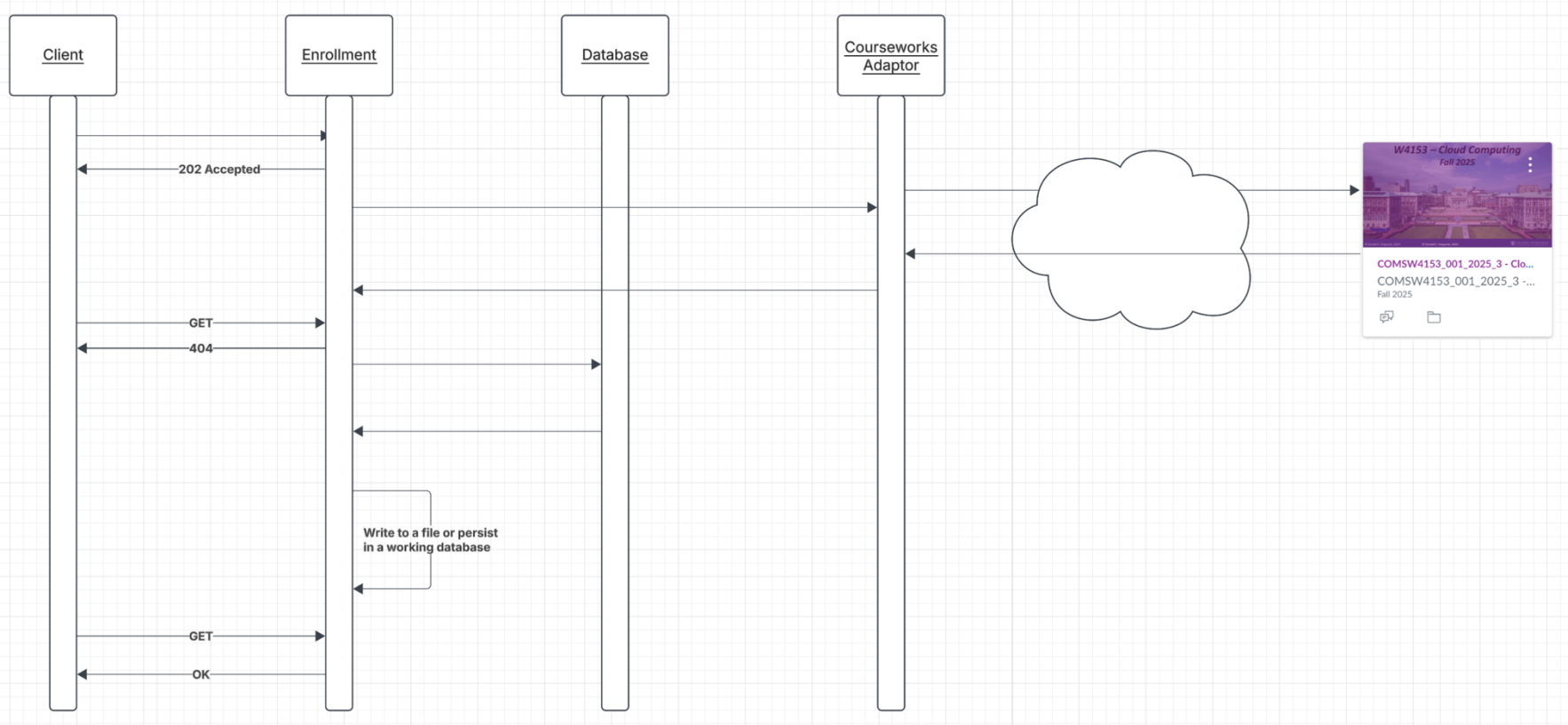
```
HTTP/1.1 201 Created
Content-Type: application/json
Location: http://example.com/users/123

{
  "message": "New user created",
  "user": {
    "id": 123,
    "firstName": "Brian",
    "lastName": "Smith",
    "email": "brian.smith@example.com"
  }
}
```

# 202 – Accepted




- Processing a request may "take a lot of time"
  - Calling other services, which call other services.
  - Complex computations
  - Complex queries or accessing multiple databases.
- While the operation is in progress, and mostly "waiting."
  - Connection are held open/in-use.
  - Base things happen when connections are held open:
    - Severs usually have a limit on the number of connections allowed → generates errors.
    - Connections, even when idle, consume resources
    - … …
- The alternate approach is to respond, "Working on it. Check back later."

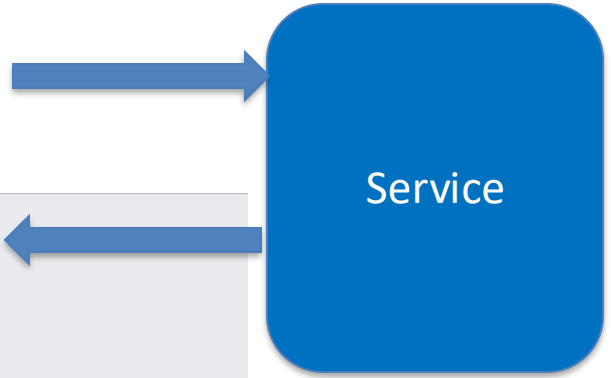
# 202 – Accepted



# 202 – Accepted

- This is a “pattern.”
- There are several design patterns for the actual message formats.
  - URL in the response body
  - Header
  - … …
- You will implement the logic several ways in your code when we implement *composition*.
- My concern from an API/REST perspective is that you choose a reasonable pattern and stick to it.

```
POST /tasks HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "task": "emailDogOwners",
  "template": "pickup"
}
```



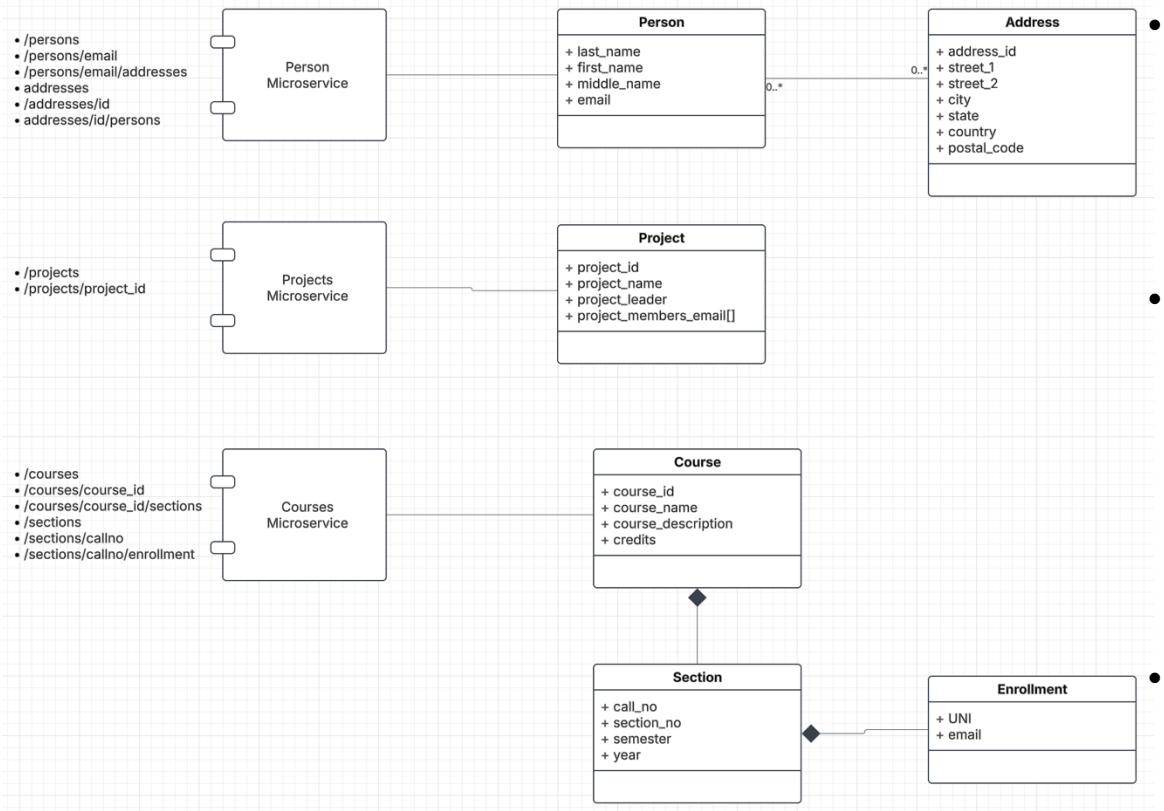

```
HTTP/1.1 202 Accepted
Date: Wed, 26 Jun 2024 12:00:00 GMT
Server: Apache/2.4.1 (Unix)
Content-Type: application/json

{
  "message": "Request accepted. Starting to process task.",
  "taskId": "123",
  "monitorUrl": "http://example.com/tasks/123/status"
}
```

# 202 Accepted

- Show /Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/sync_async
- For now, you can simply replicate the pattern to get started.
- For the next sprint, you should implement this pattern for one of your methods on a composite service (see below).

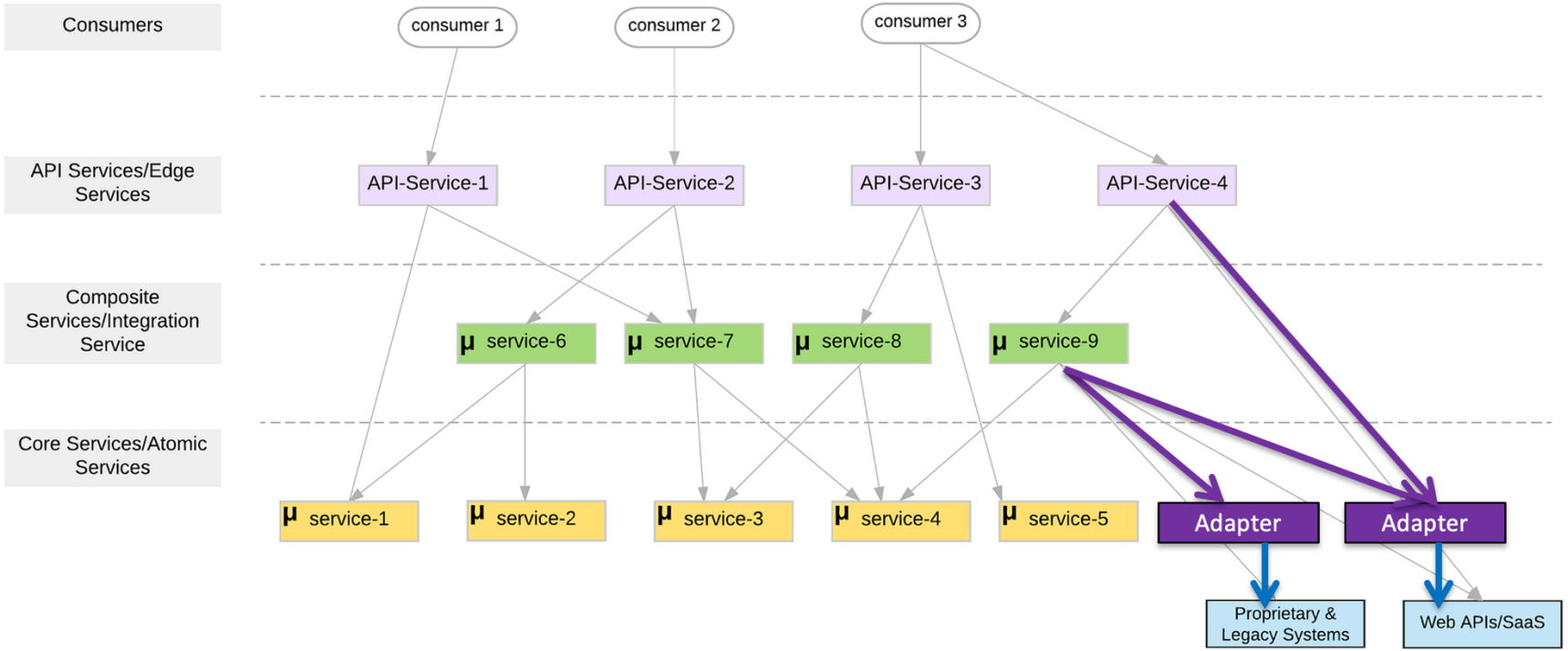
# *Composition*

# Composition – Somewhat Contrived Exampl

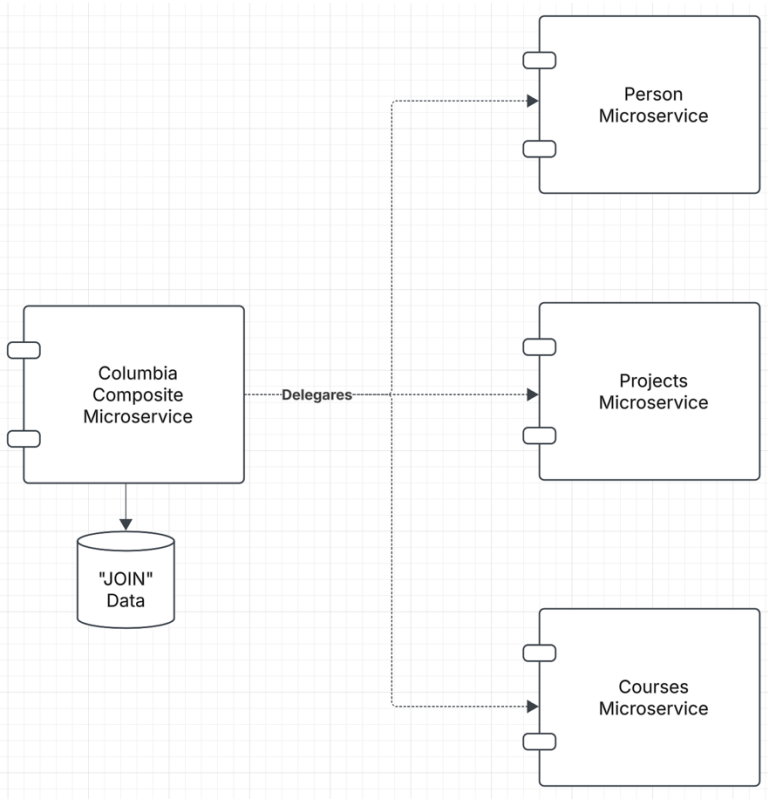



- We have 3 independently developed microservices.
  - Persons
  - Projects
  - Courses
- Our application scenarios requires *composing* them into a larger whole with paths for
  - Person ↔ Project
  - Course ↔ Project
  - Person ↔ Section
  - etc.
- And the implied integrity constraints.

# Logical Microservice Layers



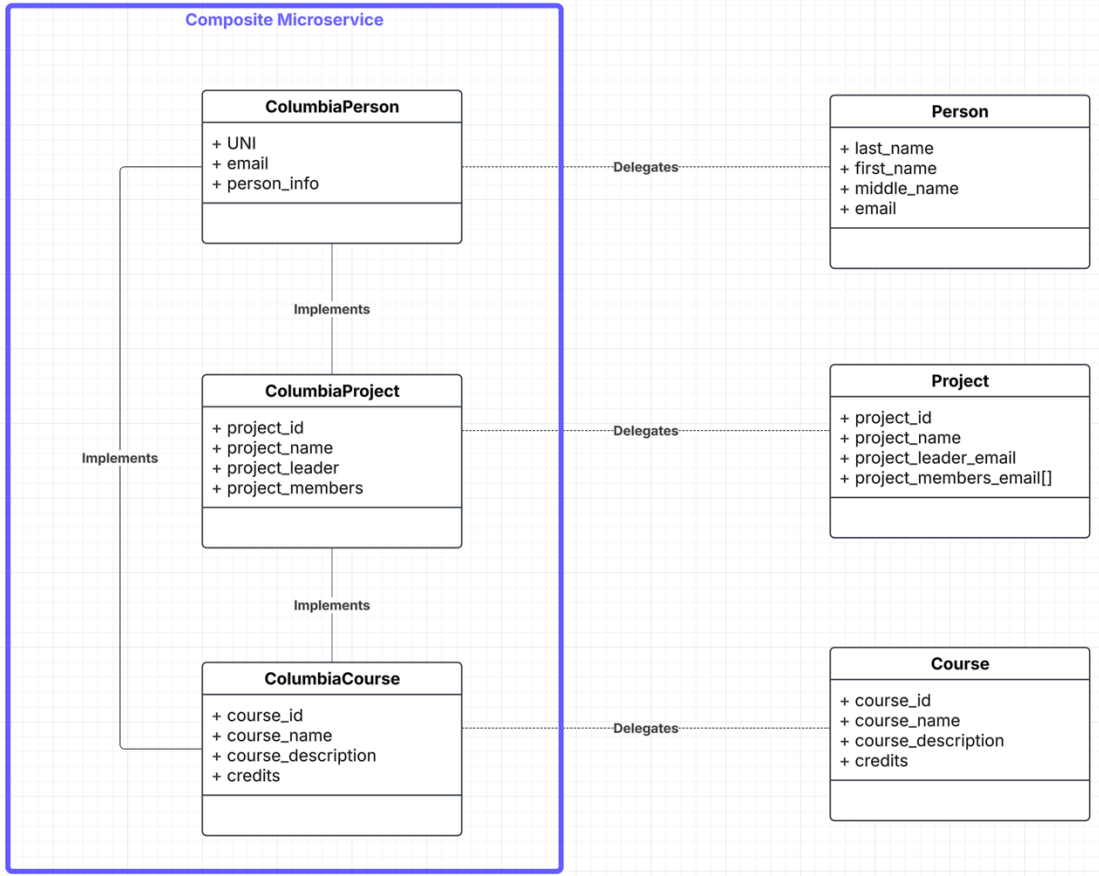
# Composite Microservice



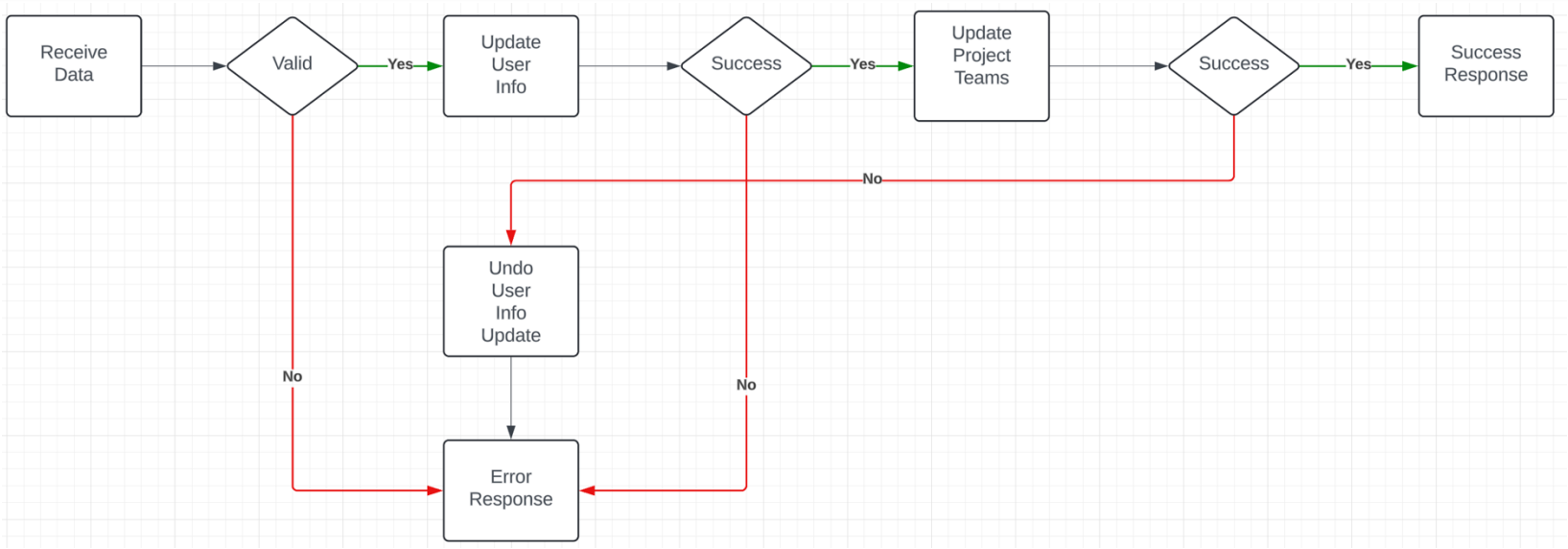

- Implements *implied links* and referential integrity between the resources.
- Maximally *delegates* onto basic microservices for the core information/behavior.
- Maintains its own database containing
  - Any additional information, e.g. UNI
  - "Associative entities" necessary to implements the implied "foreign keys" and "integrity."
- Implements "logical transactions" via undo when a failure occurs during update of a composite resource.
- This is the pattern when the basic microservices are unaware of the other basic services.

# Composite Resources and Microservice

- /columbia_persons
- /columbia_persons/uni
- /columbia/persons/uni/projects
- /columbia/persons/uni/courses
- ... ...
- ... ...
- /columbia_projects
- /columbia_projects/project_id/persons
- /columbia_projects/project_id/course
- ... ...
- ... ...
- /columbia_courses
- /columbia_courses/course_id
- /columbia_courses/course_id/sections
- /columbia_courses/course_id/sections/callno
- /columbia_courses/course_id/sections/callno/persons
- /columbia_courses/course_id/sections/callno/projects
- ... ...
- ... ...

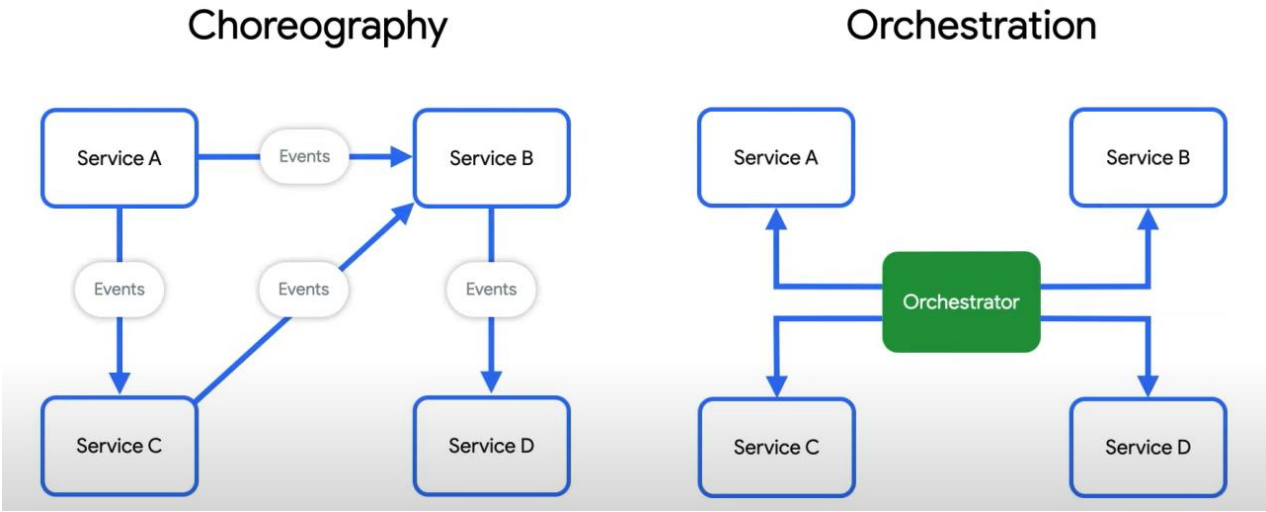


# Composite Microservice Logic




- The GET on the composite invokes GET on several atomic services. This can occur synchronously or asynchronously in parallel.
- PUT, POST and DELETE are more complex. Your logic may have to undo some delegated updates that occur if others fail.
- To get started, we will just use simple logic but explore more complex approaches.

# Behavioral Composition



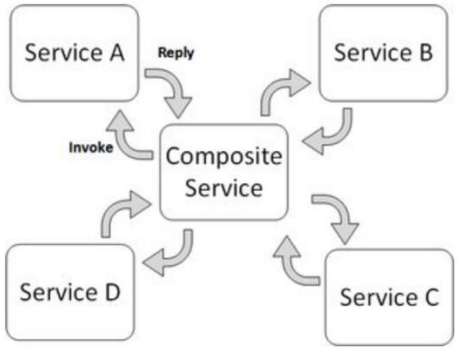

- There are two base patterns: choreography and orchestration.
- There also several different implementation technologies and choices.
- And many related concepts, e.g. Sagas, Event Driven Architectures, Pub/Sub, … …
- We will cover and explore incrementally, but for now we will keep it simple and do traditional “if … then …” code.

# Concepts

## Service orchestration

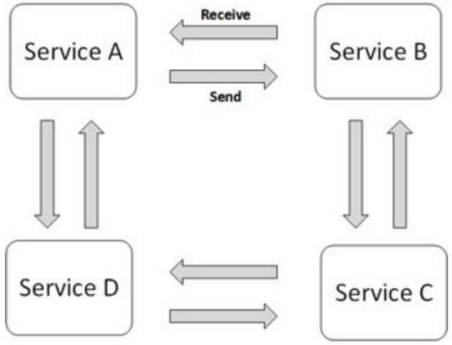
Service orchestration represents a single centralized executable business process (the orchestrator) that coordinates the interaction among different services. The orchestrator is responsible for invoking and combining the services.

The relationship between all the participating services are described by a single endpoint (i.e the composite service). The orchestration includes the management of transactions between individual services. Orchestration employs a centralized approach for service composition.




## Service Choreography

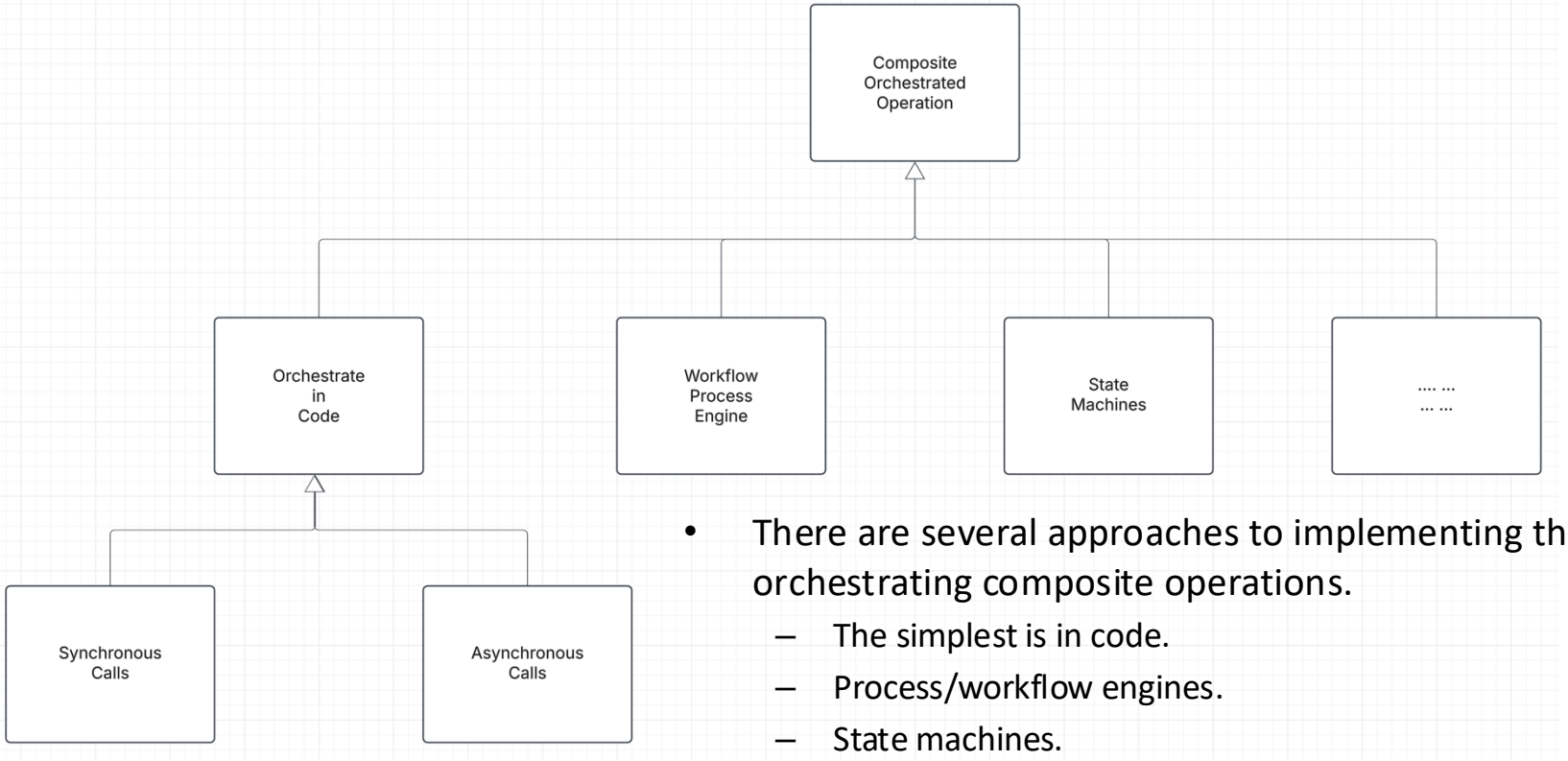
Service choreography is a global description of the participating services, which is defined by exchange of messages, rules of interaction and agreements between two or more endpoints. Choreography employs a decentralized approach for service composition.




The choreography describes the interactions between multiple services, where as orchestration represents control from one party's perspective. This means that a **choreography** *differs* from an **orchestration** with respect to where the logic that controls the interactions between the services involved should reside.

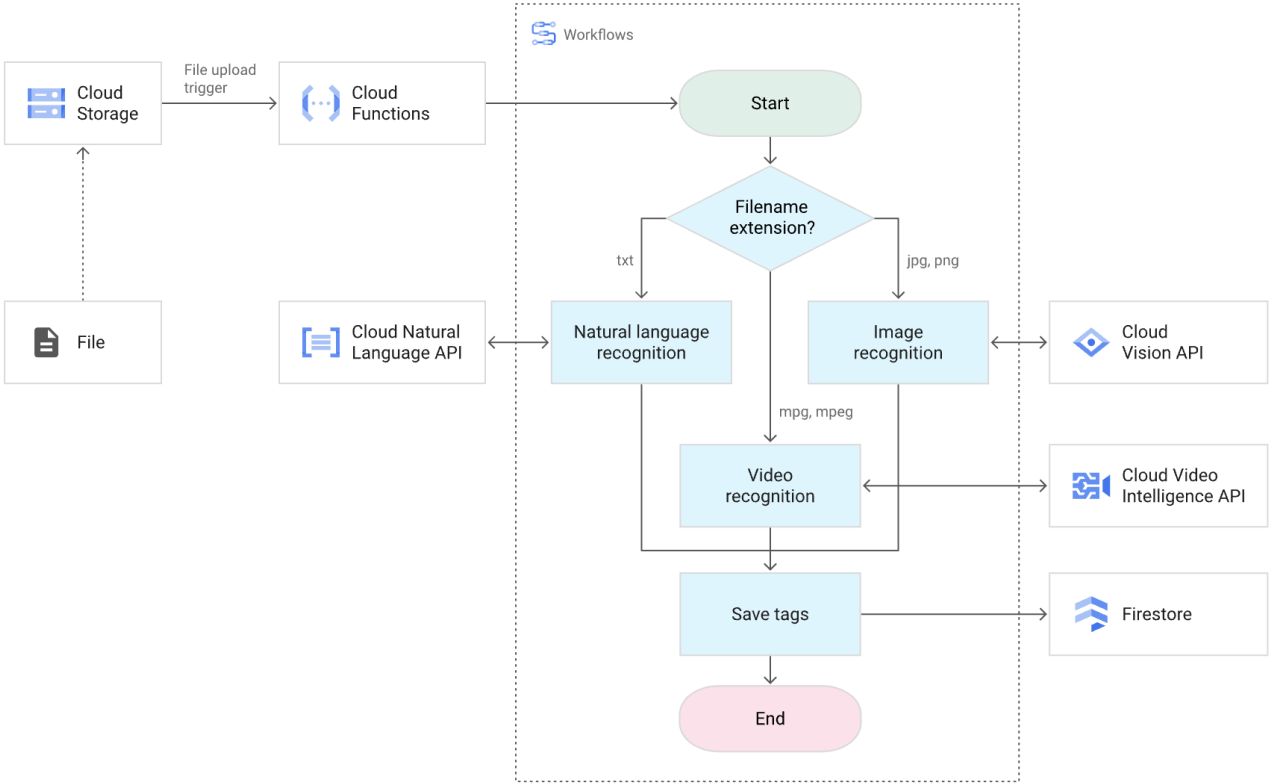
These are not formally, rigorously defined terms.

# Implementing Composite Operations

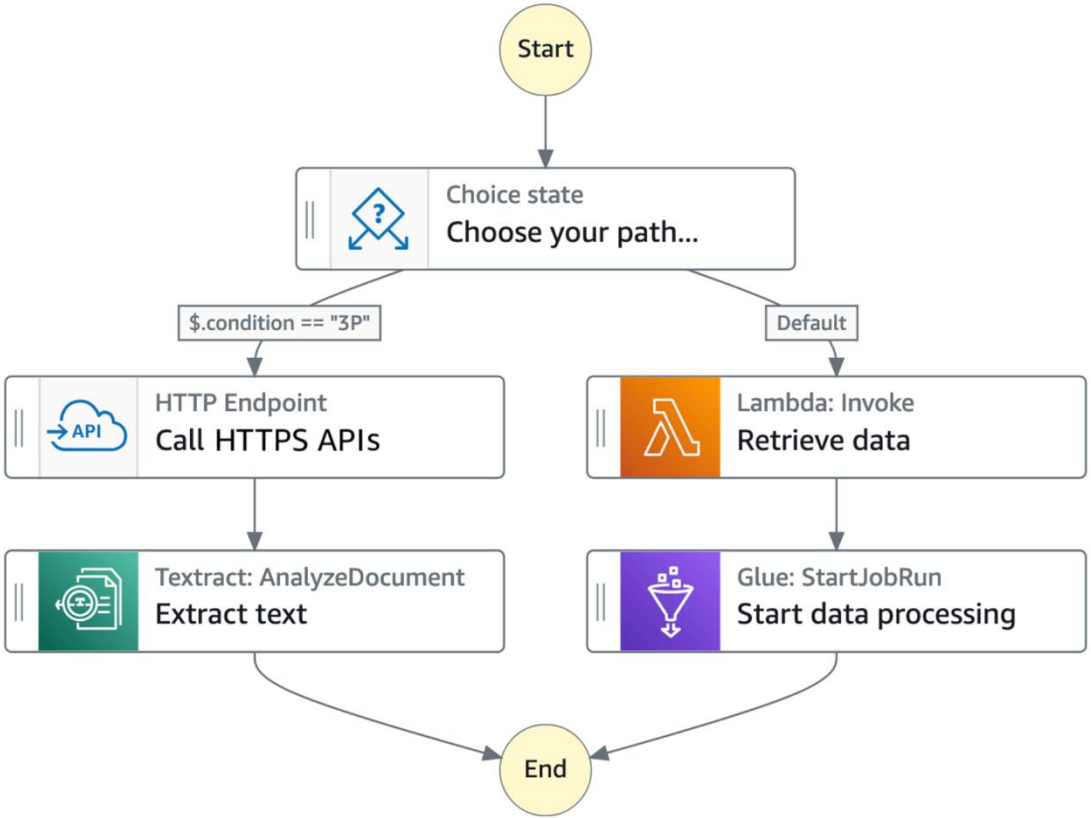



- There are several approaches to implementing the orchestrating composite operations.
  - The simplest is in code.
  - Process/workflow engines.
  - State machines.
  - … …

# Google Cloud Workflow

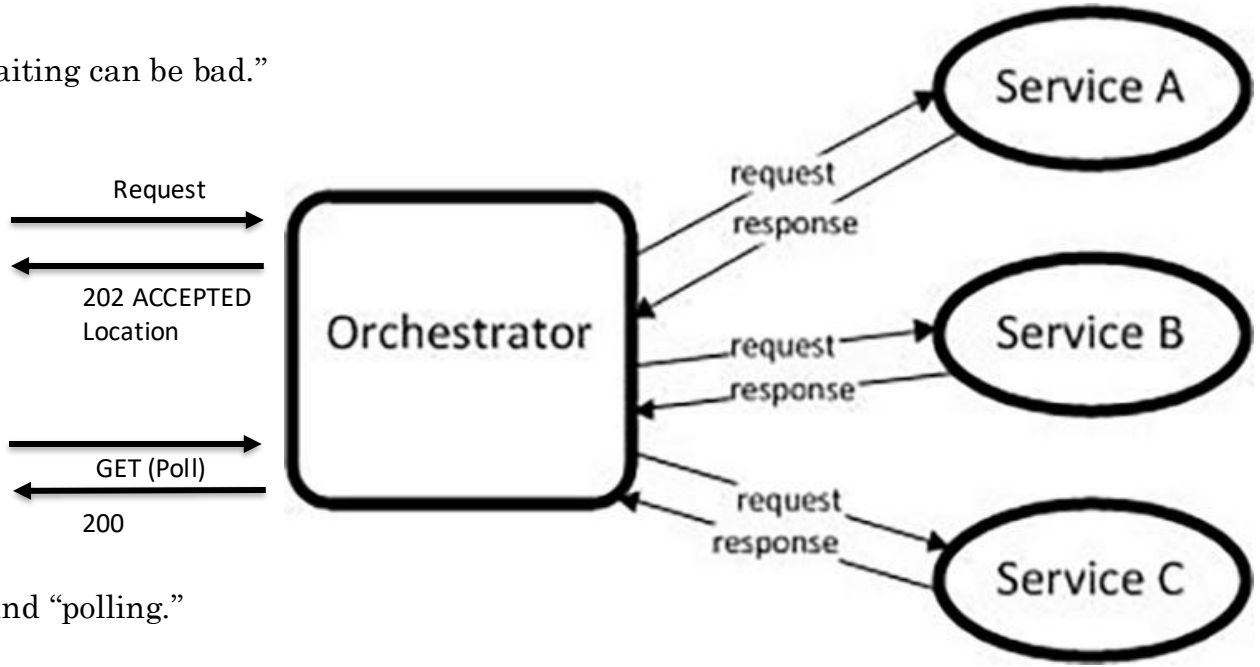


# AWS Step Functions

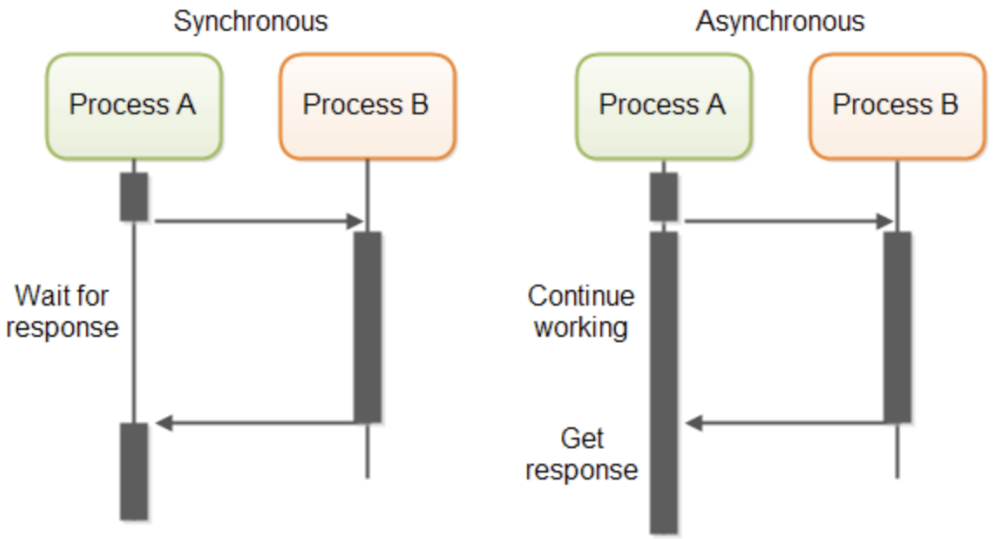


# In Code: Composition and Asynchronous Method Invocation

- Composite operations can “take a while.”
- “Holding open connections and waiting can be bad.”
  - Consumes resources.
  - Connections can break.
  - … …
- There are several options:
  - Polling
  - Callbacks
  - Asynchronous calls
  - Events
  - … …
- Our next pattern is composition and “polling.”
  Create a new user requires:
  - Updating the user database.
  - Verifying a submitted address.
  - Creating an account in the catalog service.



# In Code: Service Orchestration/Composition




RESTFul HTTP calls can be implemented in both a synchronous and asynchronous fashion at an IO level.

- One call to a composite service
  - May drive calls to multiple other services.
  - Which in turn, may drive multiple calls to other services.
- Synchronous (Wait) and calling one at a time, in order is
  - Inefficient
  - Fragile
  - … …
- Asynchronous has benefits but can result in complex code.

# Synchronous/Asynchronous Show Code

- Show example of calling URL synchronously and asynchronously
  - /Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/sync_async/async1.py
  - /Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/sync_async/sync1.py

- One of the steps in your next sprint is:
  - Build a composite microservice that aggregates your basic microservices.
  - Demonstrating one method on the composite aggregating the base services synchronously.
  - Demonstrating one method on the composite aggregating the base services asynchronously.
  - Demonstrate one method using a thread and synchronous calls, implementing 202 accepted

- You are probably confused as heck right now.
  - Think of your initial microservices as base tables.
  - Think of the composite as a VIEW representing a JOIN of the base tables.
  - Your composite microservice's code implements the basic operations, and perhaps some additional logic, in the JOIN to "compose" that basic services.

# Threads and Coroutines

Threads

- Definition: A thread is an operating system (OS)-level execution unit. Multiple threads can run truly in parallel on multiple CPU cores (if the Global Interpreter Lock, GIL, isn't a limiting factor).
- Preemptive Scheduling: The OS decides when to pause one thread and switch to another. This can happen at any point in the code.
- Context Switching: Switching between threads requires saving and restoring OS-level state, which is relatively expensive.
- Use Case: Good for I/O-bound tasks (like downloading files, reading/writing to databases, waiting on network responses). For CPU-bound tasks in CPython, the GIL prevents true parallelism—so multiple threads don't improve raw compute speed.

Coroutines

- Definition: A coroutine is a special Python function declared with async def. It represents a cooperative unit of execution managed by an event loop (usually asyncio).
- Cooperative Scheduling: Coroutines yield control voluntarily (using await) so other coroutines can run. Unlike threads, they don't get interrupted at arbitrary points.
- Context Switching: Switching between coroutines is very cheap—just a function call in the interpreter, no OS involvement.
- Use Case: Excellent for handling massive numbers of I/O-bound tasks (e.g., serving thousands of HTTP requests). Not good for CPU-heavy tasks unless you explicitly run them in threads or processes.

# For Now, … …

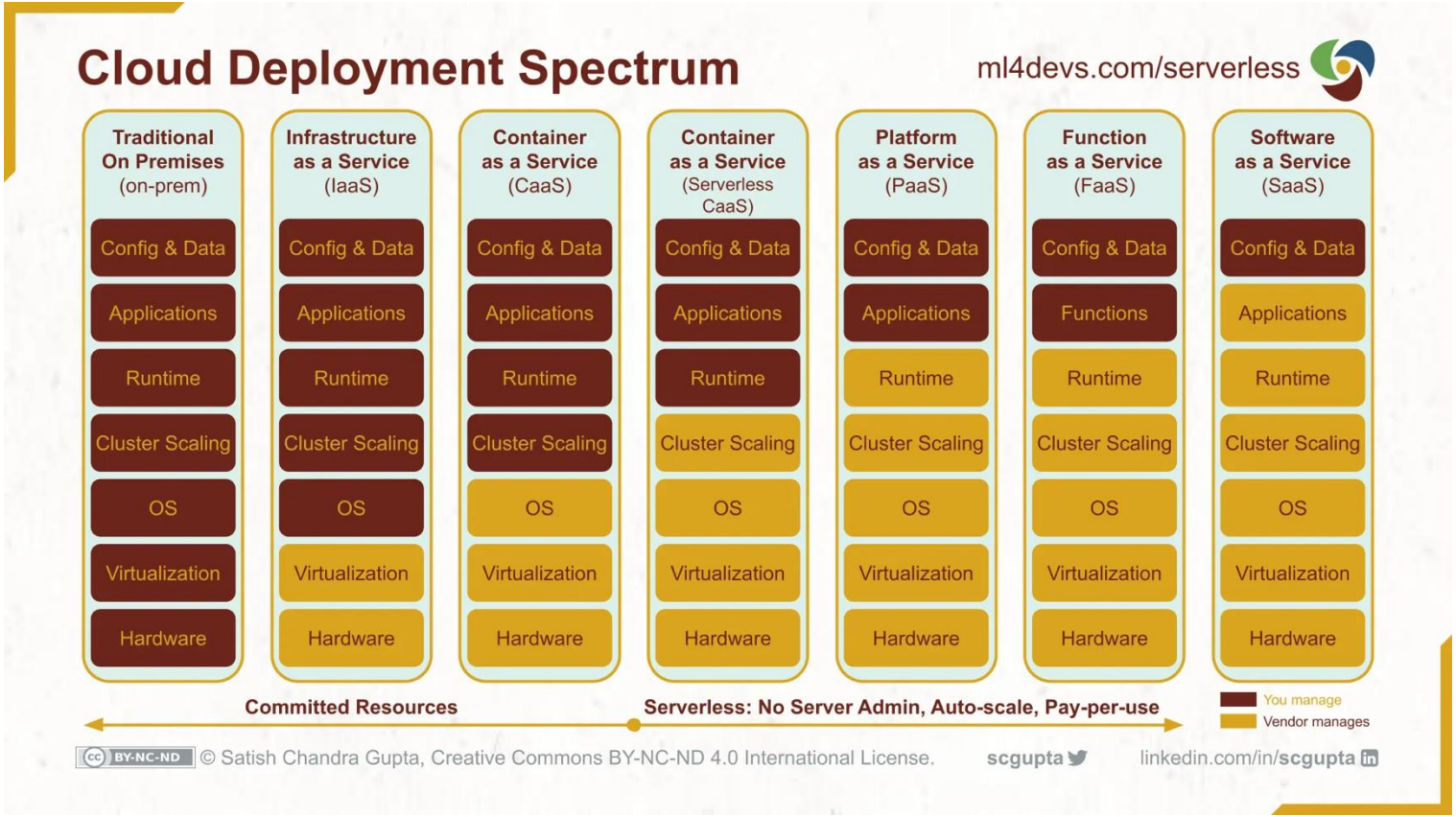
- I am trying to get you familiar with basics of asynchronous programming.
- There are much “cleaner” but potentially more complex ways to handle asynchronous and long running operations.
- We will see several examples:
  - Functions
  - Pub/Sub
  - Message Queues
  - Workflow
- But for now, just play with coroutines and threads.

~~*OAuth2*~~

~~*Open ID Connect*~~

# *Function-as-a-Service*

# Cloud Layers

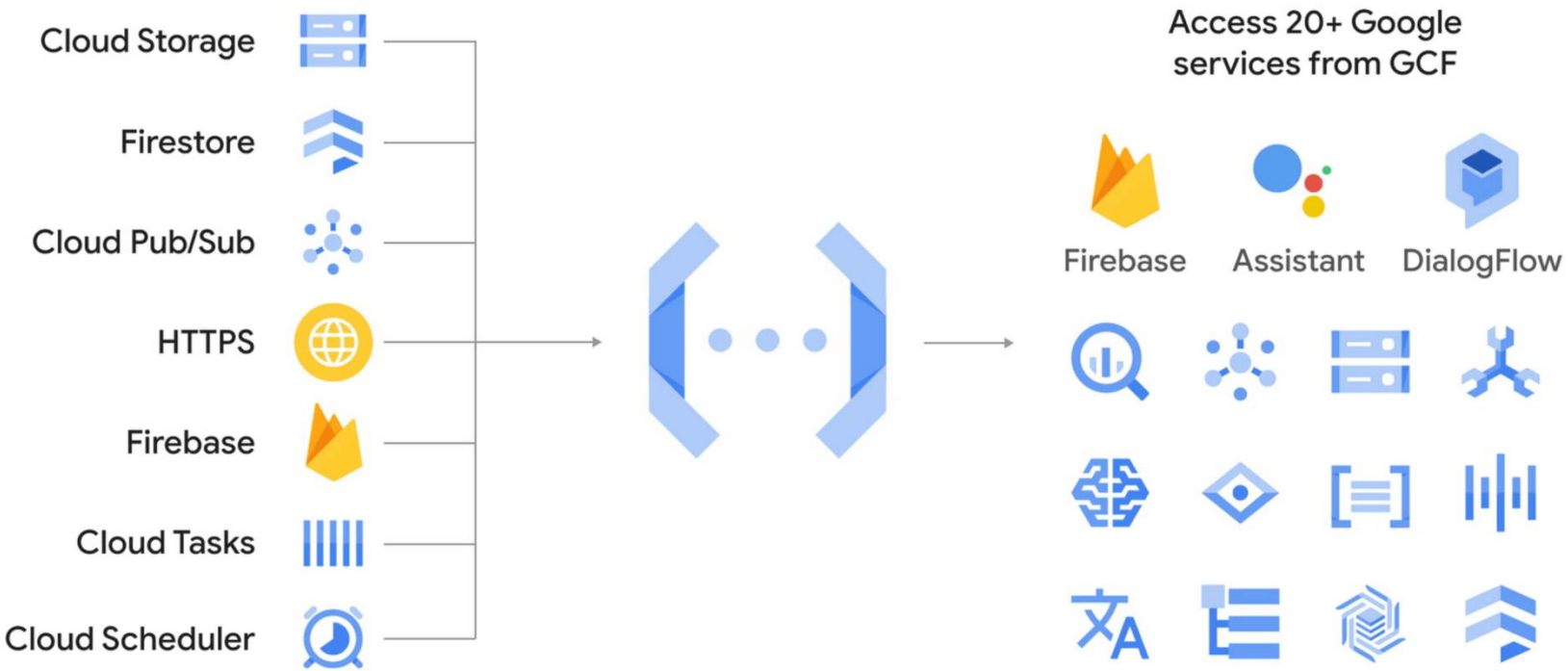


# Function-as-a-Service Principles

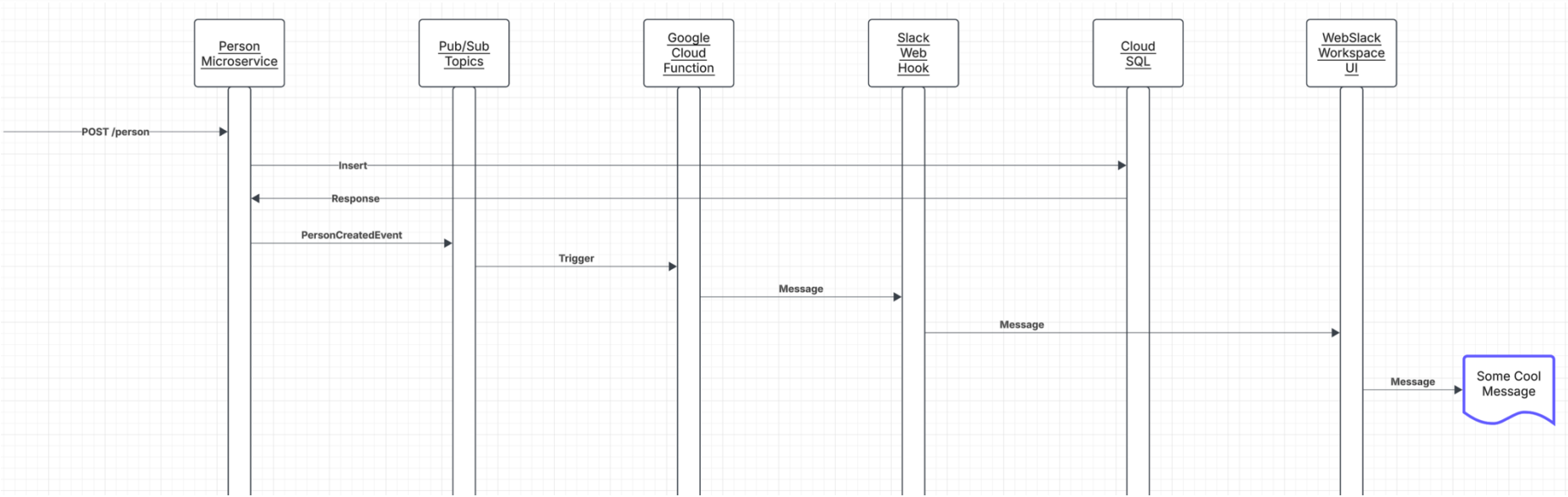
- Event-Driven Execution:
  - Specific events trigger FaaS functions. These events can be an HTTP request, a message in a queue, database update, scheduled task, … …
  - The corresponding function executes when an event occurs.
  - Think of this being like the generalization of a database trigger.
- Stateless Functions: Each function is self-contained. It maintains no state between invocations, enabling easy scaling and parallel execution.
- Short-Lived Execution: Functions run for a short duration, typically seconds to minutes. The system creates them when needed and removes them once their task is complete, optimizing resource use.
- Managed Infrastructure: The cloud provider handles the underlying infrastructure, including servers, scaling, and security. It abstracts the functional complexities away from the developers.

# Google Cloud Functions

## Cloud Functions: the glue that connects cloud services



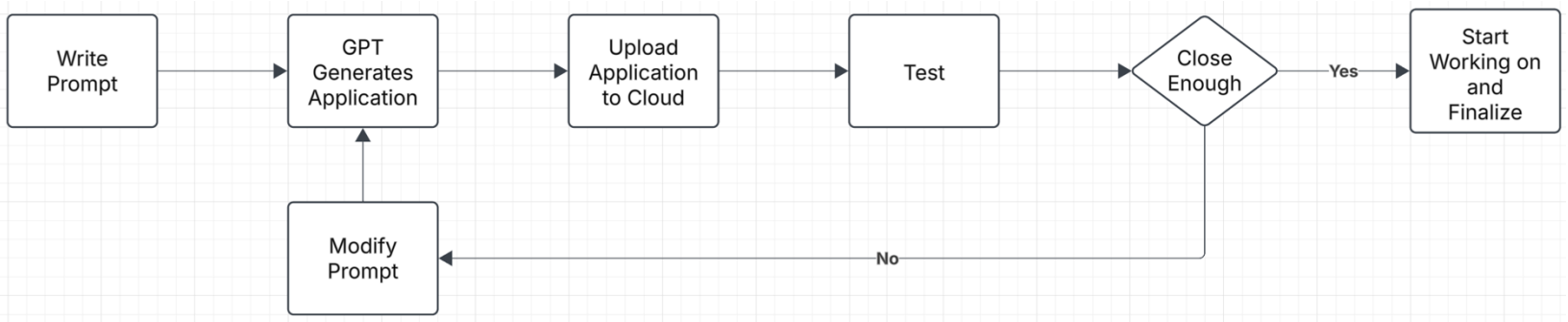
# Scenario




- For now, just focus on plumbing everything together.
- You do not need to connect the microservice. You can just trigger with gcloud CLI.
- Collaboration/Social Media space:
  - I will use either Slack or Discord. You can choose whatever you want.
  - For now, you can just write to the Google log to check and demonstrate that your function ran.

# Project Walkthrough

- /Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/simple_html
- Not tested on GCP yet. Will do this weekend, but … …
  - This is a simple ChatGPT generated template/starter project.
  - The new style of programming is … …




  - You can/should do the same.

# Show Code

- /Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/slack-pubsub-notify
- /Users/donald.ferguson/Dropbox/000/000-A-2025-Repos/pubsub-hello 2

- Where are we going with this … …
  - We are going to learn about pub/sub an events.
  - We will expand our composition example to use pub/sub and functions.
  - We will also include some form of “user notification” in our project using functions and pub/sub
  - And seeing the complexity, we will look at workflow and state machines

# *12 Factor Applications*

# 12 Factor Applications

https://dzone.com/articles/12-factor-app-principles-and-cloud-native-microser

# Dependencies

## II. Dependencies

### Explicitly declare and isolate dependencies

Most programming languages offer a packaging system for distributing support libraries, such as CPAN for Perl or Rubygems for Ruby. Libraries installed through a packaging system can be installed system-wide (known as "site packages") or scoped into the directory containing the app (known as "vendoring" or "bundling").

**A twelve-factor app never relies on implicit existence of system-wide packages.** It declares all dependencies, completely and exactly, via a *dependency declaration* manifest. Furthermore, it uses a *dependency isolation* tool during execution to ensure that no implicit dependencies "leak in" from the surrounding system. The full and explicit dependency specification is applied uniformly to both production and development.

For example, Bundler for Ruby offers the `Gemfile` manifest format for dependency declaration and `bundle exec` for dependency isolation. In Python there are two separate tools for these steps – Pip is used for declaration and Virtualenv for isolation. Even C has Autoconf for dependency declaration, and static linking can provide dependency isolation. No matter what the toolchain, dependency declaration and isolation must always be used together – only one or the other is not sufficient to satisfy twelve-factor.

One benefit of explicit dependency declaration is that it simplifies setup for developers new to the app. The new developer can check out the app's codebase onto their development machine, requiring only the language runtime and dependency manager installed as prerequisites. They will be able to set up everything needed to run the app's code with a deterministic *build command*. For example, the build command for Ruby/Bundler is `bundle install`, while for Clojure/Leiningen it is `lein deps`.

Twelve-factor apps also do not rely on the implicit existence of any system tools. Examples include shelling out to ImageMagick or `curl`. While these tools may exist on many or even most systems, there is no guarantee that they will exist on all systems where the app may run in the future, or whether the version found on a future system will be compatible with the app. If the app needs to shell out to a system tool, that tool should be vendored into the app.

- Most application frameworks have a dependency declaration concept.
- I have been showing examples with:
  - Python import
  - requirements.txt
  - etc.
- My examples also show encapsulating libraries and APIs with Python classes.
- II. Dependencies handles "the code," but we still need to handle the instance. There is a difference between, e.g.
  - pymysql
  - A specific connection.

# III. Configurations

## III. Config

### Store config in the environment

An app's *config* is everything that is likely to vary between deploys (staging, production, developer environments, etc). This includes:

- Resource handles to the database, Memcached, and other backing services
- Credentials to external services such as Amazon S3 or Twitter
- Per-deploy values such as the canonical hostname for the deploy

Apps sometimes store config as constants in the code. This is a violation of twelve-factor, which requires **strict separation of config from code**. Config varies substantially across deploys, code does not.

A litmus test for whether an app has all config correctly factored out of the code is whether the codebase could be made open source at any moment, without compromising any credentials.

Note that this definition of "config" does **not** include internal application config, such as `config/routes.rb` in Rails, or how code modules are connected in Spring. This type of config does not vary between deploys, and so is best done in the code.

Another approach to config is the use of config files which are not checked into revision control, such as `config/database.yml` in Rails. This is a huge improvement over using constants which are checked into the code repo, but still has weaknesses: it's easy to mistakenly check in a config file to the repo; there is a tendency for config files to be scattered about in different places and different formats, making it hard to see and manage all the config in one place. Further, these formats tend to be language- or framework-specific.

**The twelve-factor app stores config in *environment variables*** (often shortened to *env vars* or *env*). Env vars are easy to change between deploys without changing any code; unlike config files, there is little chance of them being checked into the code repo accidentally; and unlike custom config files, or other config mechanisms such as Java System Properties, they are a language- and OS-agnostic standard.

Another aspect of config management is grouping. Sometimes apps batch config into named groups (often called "environments") named after specific deploys, such as the `development`, `test`, and `production` environments in Rails. This method does not scale cleanly: as more deploys of the app are created, new environment names are necessary, such as `staging` or `qa`. As the project grows further, developers may add their own special environments like `joes-staging`, resulting in a combinatorial explosion of config which makes managing deploys of the app very brittle.

In a twelve-factor app, env vars are granular controls, each fully orthogonal to other env vars. They are never grouped together as "environments", but instead are independently managed for each deploy. This is a model that scales up smoothly as the app naturally expands into more deploys over its lifetime.

- Each of the deployment environments we will use has a method for configuring environment variables.
- CI/CD, GitHub actions, etc. can integrate environment variables with the development process.
- Vaults and secrets managers are a better approach for passwords.

# IV. Backing Services

## IV. Backing services

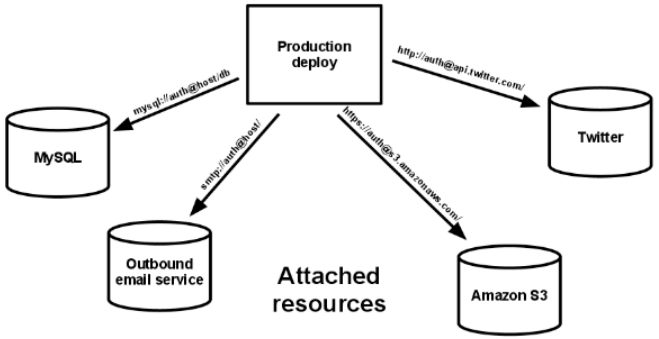
Treat backing services as attached resources

A *backing service* is any service the app consumes over the network as part of its normal operation. Examples include datastores (such as MySQL or CouchDB), messaging/queueing systems (such as RabbitMQ or Beanstalkd), SMTP services for outbound email (such as Postfix), and caching systems (such as Memcached).

Backing services like the database are traditionally managed by the same systems administrators who deploy the app's runtime. In addition to these locally-managed services, the app may also have services provided and managed by third parties. Examples include SMTP services (such as Postmark), metrics-gathering services (such as New Relic or Loggly), binary asset services (such as Amazon S3), and even API-accessible consumer services (such as Twitter, Google Maps, or Last.fm).

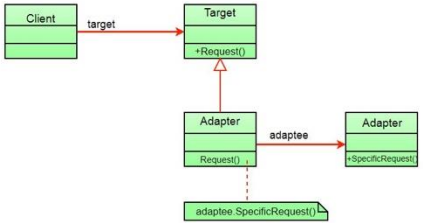
**The code for a twelve-factor app makes no distinction between local and third party services.** To the app, both are attached resources, accessed via a URL or other locator/credentials stored in the config. A deploy of the twelve-factor app should be able to swap out a local MySQL database with one managed by a third party (such as Amazon RDS) without any changes to the app's code. Likewise, a local SMTP server could be swapped with a third-party SMTP service (such as Postmark) without code changes. In both cases, only the resource handle in the config needs to change.

Each distinct backing service is a *resource*. For example, a MySQL database is a resource; two MySQL databases (used for sharding at the application layer) qualify as two distinct resources. The twelve-factor app treats these databases as *attached resources*, which indicates their loose coupling to the deploy they are attached to.




Resources can be attached to and detached from deploys at will. For example, if the app's database is misbehaving due to a hardware issue, the app's administrator might spin up a new database server restored from a recent backup. The current production database could be detached, and the new database attached – all without any code changes.

- My code has shown encapsulating backing services with a shallow adaptor.
- This is an example of the *adaptor pattern.*




- You seen examples of things like AbstractBaseDataService, … …
- The approach is also an application of several aspects of SOLID. (https://en.wikipedia.org/wiki/SOLID)

# *Next Sprint*

# Next Sprint

- Well-design REST applications for basic and c
  - eTag
  - Query parameters
  - Linked data and relative paths
  - 201 Created for POST
  - One deployed on Cloud Compute and one
- Composite microservices
  - Well-defined API
  - One operation implement synchronously a
- DBaaS and one microservice/database patt
  - 1 SQL VM/Cloud Compute MySQL instance
  - 1 or two Cloud SQL instances
- Simple Cloud Storage based web UI accessing composite, and other microservices
- Template/scaffolded Google Cloud Function triggered by pub/sub
- Updated GitHub projects and Trello for sprints

Will document.
You can accomplish in stages.